
A Contract-Grade Verifier for LLM-Generated GPU Kernels, and a Native Blackwell Backward for the Gated-Linear-Recurrence Family

Rishi Shah

Machine Learning Engineer
E3A Healthcare
rishishah994@gmail.com

Rishav Shrestha

Chief Technical Officer
E3A Healthcare
rishav@e3ahealth.com

Abstract

Systems that generate GPU kernels with language models report high correctness rates. Those rates come from a single loose test: run the kernel on a few random inputs at one fixed shape and accept it if the output is close to a reference. A kernel can pass that test and still be silently wrong. It can return an ordinary number where the true answer is a NaN or an infinity, produce a different result on each run, break the moment the shape changes, or accumulate in `fp16` where the reference keeps an `fp32` total. We build the instrument that checks correctness properly: a *contract-grade verifier* of twelve adversarial gates, each a property a correct kernel must satisfy, several of them *tolerance-free* so that no choice of threshold can explain a failure away. We aim it in two directions. Aimed outward, the verifier audits **2,638** machine-generated kernels that a public system’s own harness had already accepted as correct. It finds **39.5%** broken beyond any tolerance argument, and **62.1%** carrying at least one violation. The field’s standard test accepts **1,487** kernels the verifier rejects, against only 14 the other way. We defend the finding in four independent ways: a **7/7** positive control, a threshold-calibration sweep, **98.5%** agreement with the reference benchmark’s own correctness code, and a stratified hand-audit. Aimed inward, we test the verifier on a kernel of our own: the first native Blackwell `tcgen05` training backward for the gated-linear-recurrence (GDN) family, including the reverse-state stage the field still runs on a fallback. We establish this kernel’s correctness independently of the verifier, against a double-precision oracle, and train five family members through it, so it is a subject whose correctness we have already established. Running it through the battery is then a direct check that the battery is not tuned to flatter its author. The correctness signal behind reported progress in kernel generation is far weaker than the numbers suggest, and a set of tolerance-free contracts would close most of the gap.

1 Introduction

Generating GPU kernels with large language models has become an active research program, and its benchmarks report that a large fraction of outputs are correct and often faster than a reference [8, 5, 7, 4, 6]. Those reports are only as trustworthy as the correctness signal beneath them, and that signal is almost universally a single test: draw a few random inputs at one fixed shape and

accept the kernel if its output is close, in the `allclose` sense, to a reference. A kernel can pass this test and still be wrong in ways the test never probes. It can return an ordinary number where the reference returns a NaN or an infinity, produce a different result on each run, break the moment the sequence length or batch changes, or accumulate its reduction in `fp16` while the reference keeps an `fp32` running total. If the acceptance signal is this weak, an unknown share of the reported wins is illusory.

This paper is about that gap and the instrument that measures it. We build a *contract-grade verifier* that operationalizes the Kernel Contracts taxonomy [14]: twelve adversarial gates, each a property a correct kernel must satisfy, graded against a slow high-precision reference. Some gates need no tolerance at all and compare by exact mask or byte equality; the rest use tolerances derived from a floating-point error model rather than chosen by hand. The verifier is the spine of the paper, and we aim it in two directions.

Aimed outward, the verifier audits a public corpus of machine-generated kernels that the source system’s own harness has already accepted as correct, and measures how far those acceptances overstate correctness: 39.5% are broken in a way no tolerance can excuse. The obvious objection is that our checker is simply stricter than everyone else’s, so foreign kernels fail by construction. We answer it in four independent ways: a positive control on a kernel we can independently vouch for, a threshold-calibration sweep, 98.5% agreement with KernelBench’s own correctness code [8], and a stratified hand-audit of disputed cases. The standard by which we judge is therefore not ours alone.

Aimed inward, the same verifier judges a kernel of our own: a hand-written native Blackwell `tcgen05` training backward for the gated-linear-recurrence (GDN) family, whose reverse-state stage open implementations still run on a fallback. It is a genuine first in its own right, and we establish its correctness against an independent double-precision oracle. Running it through the verifier is then a positive control: the same battery that rejects the foreign corpus is turned on a kernel we already know is correct, and it passes. That pass answers the charge that the battery is tuned to fail foreign code. The two uses meet in the data, since the failure modes that most often sink the foreign kernels are the ones our own kernels were at risk of, and one of them is a gate that surfaced a real defect in our own code. Because the checker that rejects our failed ideas also clears a kernel we can independently vouch for, its acceptances carry weight and its rejections are diagnostic.

Contributions.

- A **twelve-gate contract-grade verifier** (Section 3) that operationalizes the Kernel Contracts taxonomy [14] at scale, with tolerance-free gates that need no threshold and thresholded gates whose bounds are derived, not guessed, and a red team of nineteen deliberately-broken kernels that demonstrates, in both directions, that the referee has teeth.
- An **at-scale rigor-gap audit** of 2,638 accepted kernels (Section 3): a tolerance-free floor of **39.5%**, with **62.1%** carrying a contract violation. Four independent defenses hold the finding up: a 7/7 positive control, a threshold-calibration sweep, **98.5%** agreement with KernelBench’s own correctness code, and a stratified hand-audit. A differential against the standard test shows that it certifies **1,487** broken kernels, against 14 in the reverse direction.
- The **first native Blackwell `tcgen05`/tensor-memory training backward for the GDN family** (Section 4), including the reverse-state stage others keep in Triton. It is verified to 3.3×10^{-3} against an `fp64` oracle, it is bit-for-bit deterministic, it trains five family members through one backward, and it handles correctly the tensor-memory constraint behind the open #904 bug.
- **Supporting results and methodology** (Appendix C): five further results, among them six contract-verified Mamba-3 kernels that bypass #904 on Blackwell, a 1.1B-parameter model trained end-to-end through the C5 fused-block path, and a from-scratch GRPO system that autotunes them, each reported with the number behind it.

We interpret and stress-test these results in Section 5 and state their limitations. Every quantitative claim is backed by a committed artifact under a fixed software environment, and a number resting on an unreleased intermediate is marked at the point of use. Single-row reproduction of any audited kernel requires only the public verifier and the public MIT-licensed corpus.

Code and data availability. Everything this paper is built on is public at <https://github.com/RishiShah99/lethe> under the MIT license: the verifier and its twelve gates, the nineteen deliberately-broken kernels of Section 3, the audit harness, the six Mamba-3 Triton kernels, the native Blackwell backward, the GRPO trainer, and the JSON evidence behind every number reported here. The audit ships as its own per-row record, one line per audited kernel carrying the corpus identifier, operator class, run status, recorded speedup, and the verdict and reason of every gate that fired. Each of the three selection filters, the 2,638 denominator, the per-gate counts, and the tolerance-free floor is recomputable from that one file, and its identifier column joins directly against the corpus, so any individual row cited here can be pulled and re-run. Two things that back numbers in this paper are *not* in that per-row file: the external check’s own per-row verdicts, which the differential of Section 3.4 and the 98.5% agreement of Section 3.3 are joined against, ship only as aggregated cell counts and a disagreement list; and the aggregation predicates are described in Appendix B.1 rather than shipped as code. The audited corpus is the public MIT-licensed `hkust-nlp/drkernel-coldstart-8k` release [7], pinned at revision `cba0ef0`; the audit ran on a B200 under torch 2.12 / triton 3.7, and the harness is public, so the run reproduces on any B200 under that pin.

2 Background and Related Work

Generating GPU kernels, and how their correctness is judged. A line of work trains or prompts language models to emit GPU kernels that replace a reference operator, and grades them on suites such as KernelBench [8] and TritonBench [5], or on system-specific corpora such as Dr. Kernel / KernelGYM [7] and the Sakana AI CUDA Engineer archive [9]. Across these, correctness is decided by an approximate equality check (`torch.allclose` at `atol=rtol=10-2` in the common KernelBench setting) over a handful of random inputs at a single fixed shape. This is inexpensive and reproducible, but it probes a narrow slice of a kernel’s behavior.

The correctness illusion, and how our work differs. The observation that loose checks accept broken kernels is not new. Sarkar [10] argues the point on a hand-built set of twenty-four kernels but reports no rate on any real accepted corpus. A companion paper [11] attacks the threshold rather than the benchmark, calibrating a per-(operator, dtype) `atol` empirically as the 95th percentile of the error observed over passing runs of the *correct* kernel, scaled by a 1.5 safety factor. It is the strongest existing form of the “just tighten the tolerance” answer, and its own numbers mark that answer’s limits: detection rises from 1,805 of 2,467 (73.2%) to 2,034 of 2,467 (82.4%), while false positives on correct controls rise from 0 of 1,882 to 20 of 1,882, and the evaluation is on author-seeded variants rather than on kernels any system accepted. Three differences follow. Their threshold is *fitted*, so judging an operator first requires a corpus of known-correct runs of that operator; ours is derived analytically from how rounding error accumulates over a reduction, so it is available for an operator seen once. Their gain is bought with false positives on correct kernels, a cost a tolerance-free gate cannot incur because it has no threshold to miscalibrate. And a calibrated `atol`, however tight, still compares magnitudes, so a swallowed NaN, a nondeterministic output, and an aliased buffer lie outside what any tolerance can express. We read the two lines as complementary: calibration sharpens the thresholded gates, and is exactly why we put the tolerance-free ones under the load-bearing claim.

The nearest prior *instrument* is `robust-kbench` [4], which reaches the same diagnosis we do, that existing kernel benchmarks carry exploitable loopholes and too narrow a set of testing conditions, and answers it with a harder benchmark and an agentic pipeline whose correctness filtering is performed by *language-model verifiers*. We differ in the instrument, not the diagnosis. A verifier that is itself a model returns a judgement, so its false-accept rate is a property of that model on that kernel and must be established separately; ours is twelve fixed gates whose verdicts are exact mask, byte, or derived-bound comparisons against the reference operator, with no model in the loop and, on the tolerance-free channels, no threshold either. That is what lets us state a rate on a corpus someone else has already accepted, and lets a reader check that rate without adopting our model.

A separate strand documents *performance* loopholes, kernels that game the timing harness by reusing an output buffer, returning a lazily-evaluated tensor, or exploiting a fixed input, in the Sakana AI CUDA Engineer [9] and CUDA-L1 [6] systems, with a companion line of work hardening harnesses against such exploits [20]; this is an axis orthogonal to the numerical silent-wrongness we measure.

The Kernel Contracts taxonomy [14] enumerates correctness classes for GPU kernels. That taxonomy is theirs and we cite it as such; what is ours is the first runnable, at-scale operationalization of it, to the best of our knowledge. Our distinct contributions over this literature are a *tolerance-free* correctness measure that needs no approximate-equality threshold, a quantified audit on thousands of *accepted*, *real* kernels, and a certified native artifact the same verifier passes.

The gated-linear-recurrence family. Sub-quadratic sequence models replace attention’s quadratic cost with a fixed-size recurrent state. Mamba [2] and its structured-state-space-duality successor Mamba-2/SSD [1], gated linear attention (GLA) [17], DeltaNet [18] and its gated variant [16], and Mamba-3 [3] all instantiate one *gated linear recurrence*. As we make precise in Section 4, each is a fixed setting of that recurrence’s gates rather than a different recurrence, so a single backward differentiating the general form yields the training gradients for all of them. The training backward is dominated by two hard stages, a reverse-time inter-chunk state scan and a WY / triangular-inverse vector-Jacobian product, and the reference open-source implementation of both is the Triton *flash-linear-attention* (fla) library [19], which is our speed baseline.

Blackwell tcgen05, tensor memory, and #904. NVIDIA’s Blackwell generation (the B200, architecture sm_100) adds a fifth-generation tensor core, *tcgen05*, whose matrix-multiply operands reside in a new scarce on-chip space, Tensor Memory (TMEM), governed by a hard 512-column budget per warpgroup. A multi-GEMM kernel must manage TMEM allocation and release lifecycles by hand; a lifecycle error yields illegal machine code or a deadlock. This constraint is the crux of the open issue *state-spaces/mamba#904* [13]: the official Mamba-3 backward uses the tensor-core *matmul* (`tl.dot`), and at the normal performance setting a compiler pass requests 544 TMEM columns against the hardware’s 512, so the kernel fails to compile at those settings and falls back to a path the issue reports as 38.7× slower on GB200. That issue attributes the slowdown to a `ptxas C7907` error eliminating autotuner configurations; we reproduce the tensor-memory half of the failure on our own stack and report the C7907 half as *not* reproduced there (Appendix C.2), and the 38.7× figure is theirs rather than one we re-measured. The tensor-memory half is addressed by a merged Triton change (PR #9093) that has not shipped in any Triton 3.7.x release, so a pinned pre-#9093 stack still reproduces it. Both of our headline contributions also engage this constraint: the verifier’s RES-02 gate detects it, and the native backward gets the lifecycle right where the official kernel does not.

3 A Contract-Grade Verifier and the Rigor-Gap Audit

We first describe the verifier and its fairness properties, then the audit and its four independent defenses, then the differential against the benchmark’s own harness, and finally the robustness of the finding across a second stack and a second corpus.

3.1 The twelve gates

A kernel that passes a single random-input, fixed-shape check can still be wrong on extreme inputs, wrong at a different shape, nondeterministic, secretly low-precision, or wrong about non-finite values. The verifier therefore asks twelve questions, each an adversarial contract graded against a slow reference (Table 1). For our own operators the reference is a plain high-precision loop that is *defined* to be ground truth and raises rather than execute in reduced precision, so that it cannot be misused as a candidate. In the outward audit the reference is the corpus row’s own `fp32` PyTorch module, held to the same role by the mixed-precision convention of Section 3.2: for a half-dtype probe it consumes inputs and parameters round-tripped once through that dtype and then computes in `fp32`, so the answer key is never itself a reduced-precision computation.

Two design choices make the battery rigorous. First, every tolerance is derived rather than chosen: ORD-01’s bound follows how floating-point error actually accumulates over a reduction, $\text{atol} \approx 4 \varepsilon \sqrt{N} \cdot \text{scale}$, so a correct kernel passes with wide margin and a wrong one misses by orders of magnitude, and every knob is set at the loosest point that still sides with the candidate. Second, the input draw is seeded, so a verdict depends on the kernel and not on luck. We state the exact mechanism because the two entry points differ. Called directly, as the red team and our own kernels’ acceptance gates call it, the battery re-seeds from a fixed value per gate, mixing in the gate’s name so

Table 1: The twelve contract gates, operationalizing the Kernel Contracts taxonomy [14]. Gates in the *tolerance-free* group (EXC-01, EXC-02, ORD-02, plus aliasing and crash detection) compare by exact mask or byte equality and admit no tolerance argument; the remaining gates use scale-aware derived tolerances. The reported tolerance-free floor drops EXC-02 from that group and adds one channel that is not a gate at all, a non-artifact failure to run; Appendix B.2 gives the exact channel list and the effect of including EXC-02.

Gate	Property checked
CMP-01	correct on many random <i>and</i> adversarial inputs (zeros, 10^6 , 10^{-6} , denormals, long L)
CMP-02	the <i>gradients</i> are correct, not only the outputs (autograd versus finite difference)
CMP-03	correct across shapes (batch, length, width), not only the one tested
ORD-01	reordered summation stays within a derived $\propto \sqrt{N}$ rounding bound
ORD-02	byte-for-byte identical across five repeats, and does not alias a shared buffer
ORD-03	correct on an input constructed to expose a bad summation order
PRC-01	correct in <code>fp32</code> , <code>fp16</code> , and <code>bf16</code>
PRC-02	fed <code>fp16</code> , still keeps an internal <code>fp32</code> running total
EXC-01	fed an input with NaN and signed infinities scattered at known positions, the output’s non-finite positions and signs match the reference exactly
EXC-02	flush-to-zero handling of subnormals matches the reference
RES-01	output lives on the same device as the input
RES-02	the compiled kernel fits real hardware limits (registers, shared memory, TMEM budget)

that no two gates see the same stream. The corpus audit instead fixes one seed per worker process and lets the draws run in gate order, which is what the released verdicts were frozen against; that makes a verdict a deterministic function of the (reference, candidate) pair and the gate order, but not of the gate in isolation. On a forward-only inference corpus, CMP-02 (gradient) has no autograd graph to check and RES-02 (resource metadata) has no compile-time metadata to read, so ten of the twelve gates carry the audit. Of those ten we treat seven as *load-bearing*: CMP-01/03, ORD-01/02, EXC-01/02, and PRC-01. The other three are set aside for stated reasons rather than by fail rate. ORD-03 is the deliberately-relaxed backstop whose coverage is carried by CMP-01 and ORD-01; PRC-02 is not a band-gate and is graded by a separate accumulator mechanism; and RES-01 is low-signal at 1.8%. The two idle gates are not idle on our own kernels: the committed test suite runs the full twelve on them, supplying compiled-kernel resource metadata so RES-02 is applicable and per-gradient views so CMP-02 has an autograd graph to differentiate. The positive control of Section 3.3 runs the same ten-gate audit subset as the corpus, by design, so that the control and the corpus are judged by identical machinery.

To show the verifier has teeth, the repository ships nineteen deliberately-broken kernels, each embodying one trick: returning its input, caching an answer, accumulating in `fp16`, behaving nondeterministically, flipping an infinity to a finite value, working at only one shape, or silently moving data to another device. A test asserts that the verifier rejects every one *and* clears the correct reference. Catching all nineteen while clearing the correct operator is the two-sided check that makes the referee trustworthy before it is ever aimed at foreign code.

3.2 The audit

We ran the verifier over Dr. Kernel / KernelGYM [7] (`hkust-nlp/drkernel-coldstart-8k`, MIT license), a public corpus of 8,920 supervised-fine-tuning trajectories each ending in a Triton kernel that replaces a PyTorch module in the KernelBench Model/ModelNew convention. This corpus was the richest auditable release among the systems we surveyed. Auditing its SSM-adjacent operator classes (matmul, attention, softmax, scan, norm, conv, reduction) yields **3,134** kernels, each run on a B200 (torch 2.12 / triton 3.7) in its own sandboxed subprocess. Three filters then reduce that set to the headline denominator, applied in this order. First, **2,970** carry a positive recorded speedup (`final_speedup > 0`), which is the corpus’s own record that the kernel cleared its pipeline and was timed. The release ships no separate correctness column, so this is the strongest acceptance predicate the artifact supports; Section 3.3 reports one sampled row where a speedup was recorded for a candidate whose output our harness could not read at all, which bounds how much that predicate should be read as a correctness certificate. Second, **2,919** of those expose a floating-point tensor input

for the gates to perturb; the 51 that do not are marked not-applicable rather than counted as failures. Third, **2,638** remain after excluding toolchain and compile artifacts. This accepted-only set is the headline denominator. Appendix B.1 gives the operator classifier and the artifact predicate exactly, so each of the three counts is reconstructible.

Fairness is enforced in code by five rules, pinned by nineteen unit tests. Inputs that the *reference* itself cannot run are marked not-applicable and never counted as candidate failures; tolerances sit at the loosest defensible point; positions at which candidate and reference agree on a NaN or an infinity are not charged as value mismatches, since a non-finite admits no tolerance comparison, while a *swallowed* non-finite is still caught by EXC-01; a candidate is charged only for its own declared compute precision; and the mixed-precision and shape conventions are fixed in advance. These rules move every ambiguous case toward the candidate. One precondition cuts the other way. EXC-01 probes behavior on inputs a kernel’s author may never have intended to support, since it scatters non-finites into the input rather than waiting for one to arise. We take the reference to define the contract: every kernel in this population is advertised as a drop-in replacement for one specific PyTorch module, so if that module propagates a non-finite and the replacement returns an ordinary number instead, the operator’s behavior has changed, whether or not the change could be argued to be an improvement.

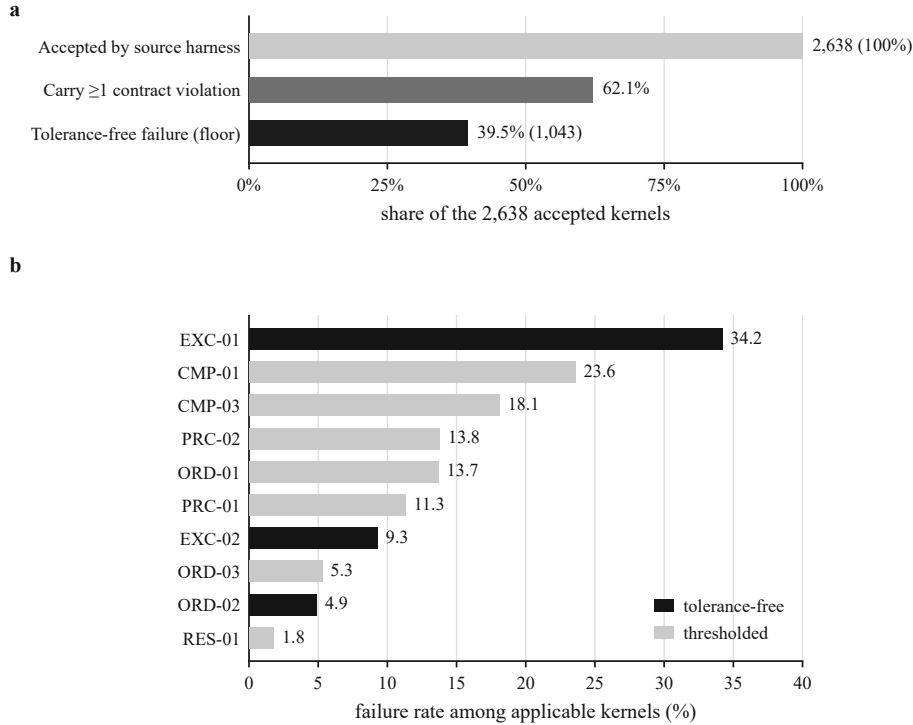


Figure 1: The rigor-gap finding. *Top:* of the 2,638 kernels a public system’s own harness accepted as correct, 62.1% carry at least one contract violation and 39.5% (1,043) fail a tolerance-free gate, the floor no tolerance argument can reach. *Bottom:* per-gate failure rate among the kernels where each gate applies (tolerance-free gates in solid black). The modal defect, EXC-01 non-finite non-propagation, is the failure mode that turns a training-time error into silent data corruption. Per-gate rates use per-gate applicability denominators and are therefore not additive to the corpus-level floor above; exact counts are in Appendix B.

Headline. Among the 2,638 accepted kernels, **62.1%** carry at least one contract violation, and **39.5%** (1,043 of 2,638) fail a *tolerance-free* gate, meaning they are broken in a way no tolerance argument can excuse. Figure 1 presents the finding as an acceptance funnel over the 2,638 kernels together with the per-gate failure rates (exact counts in Appendix B). The modal defect is a kernel silently replacing a NaN or infinity with an ordinary number (EXC-01), the failure mode that converts a training-time error into silent data corruption. That channel carries the floor: 868 of the 1,043

floor members fail EXC-01, and the floor without it is 229 kernels (8.7%). The floor is therefore a non-finite-propagation result, and a floor spread evenly over five unrelated channels would say less, not more. Breadth is carried by the 62.1%, and it survives the same ablation: EXC-01 accounts for 868 of the 1,639 kernels carrying a violation, and removing the gate entirely leaves **42.9%** (1,132 of 2,638), above the tolerance-free floor this paper leads with. No other gate moves the figure by more than five points (Table 4). The 62.1% is computed over gate verdicts and run failures, and output aliasing is tracked outside that union even though it is a floor channel, so nine aliasing-only kernels sit inside the 39.5% and outside the 62.1%. Folding aliasing into ORD-02, where Table 1 already defines it to belong, gives 1,648 of 2,638 (62.5%) and makes the floor a strict subset; we report the smaller number, so the headline is the conservative reading in both directions.

Both figures are robust to aggressive re-slicing. Adding CMP-03 shape rejection to the floor raises it to 41.1%, a further 40 kernels: most shape breakage is a value mismatch at the new shape, which carries a tolerance argument and is therefore excluded from the floor, and only outright rejection of the shape is not. The floor’s one run-failure channel, the 62 kernels that fail to execute for a reason our artifact filter does not catch, is not carrying it either. Some of those 62 are toolchain-shaped in a way the four-prefix artifact predicate misses. Deleting the channel outright leaves 981 of 2,576, or 38.1%. The 62.1% violation rate survives stripping the entire matmul class, the worst case for the TF32 tolerance argument, at 61.9%, and survives stripping every gate to which a tolerance argument could be attached at all (ORD-01, PRC-01, PRC-02, EXC-02) at 54.7%.

3.3 Four independent defenses of the finding

We return to the objection raised in Section 1, that the checker is simply stricter than everyone else’s and foreign kernels therefore fail by construction. We answer it in four ways, each falsifiable on its own.

(1) Positive control: 7/7. Our own six Mamba-3 Triton kernels (Appendix C.1) and the native GDN backward (Section 4) run through the *same* battery in the *same* Model/ModelNew convention. All seven pass every applicable gate and are clean on every tolerance-free channel, the exact classes that sink the foreign corpus. One of the seven is independent by construction. The verifier’s tolerance model was calibrated against those six Triton kernels, whereas the native GDN backward played no part in calibration, so it is a control the thresholds were never fitted to. The tolerance-free channels carry no threshold to calibrate in any case, which makes a clean result on those channels calibration-independent for all seven. The control caught a gap in our own code. C5 (`fused_block_forward`) inferred its channel dimension from the input tensor without checking it against the convolution-weight channels, a missing input-validation guard rather than a wrong-answer bug. Commit 7f46226 adds the guard in three files: the two public ops, and the *reference* as well, since a depthwise convolution with mismatched channels is not this block’s math on either side. Guarding the reference is what changes C5’s CMP-03 verdict from a failure to not-applicable, under the same reference-applicability rule applied corpus-wide, and that is the convention rather than an exception: six of the seven control kernels resolve CMP-03 as not-applicable for the same reason. The control then reads 7/7, and the per-gate applicability behind that figure is uneven: nine gates apply to all seven kernels and pass on all seven (CMP-01, ORD-01/02/03, PRC-01/02, EXC-01/02, RES-01), while CMP-03 applies to one. What is load-bearing here is that the gate flagged the author’s own kernel at all, and that we changed the code on both sides rather than re-labelling the verdict on one.

(2) Threshold calibration. This is the graded version of the positive control. For each band-gate on a known-correct operator, we inject a perturbation of known magnitude and locate the exact pass-to-fail trip point, bracketed between the noise floor (the error a correct kernel emits) and the real-error onset (the error a wrong kernel emits). Every band-gate shows clean separation (Figure 2; the precise margins are tabulated in Appendix B). One margin is thin: ORD-03 sits only 1.2× below its real-error onset because it is the deliberately-relaxed gate, and its coverage is carried by CMP-01 and ORD-01.

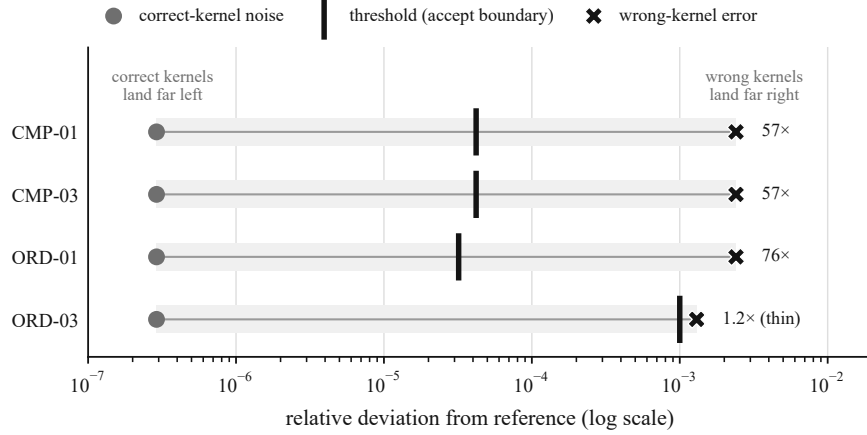


Figure 2: Threshold calibration. For each band-gate, the chosen threshold (bar) sits inside the safe margin between the noise a correct kernel emits (circle, left) and the error a wrong one emits (cross, right), on a log scale of relative deviation from the reference. The thresholds are neither too tight, so a correct kernel is never flagged, nor too loose, so a wrong one is always caught. ORD-03’s upper margin is thin (1.2×). PRC-02 is graded by a different mechanism (an fp16-carry accumulator crosses its threshold for $N \geq 2048$) and is omitted here.

(3) The benchmark’s own harness agrees (98.5%). To close the objection that our replica of the standard check is not faithful to it, we ran KernelBench’s *own* correctness code [8] at the pinned commit 48642c5c on the accepted pairs and joined *their* verdict against our reimplement. Agreement is **98.5% over 1,030 pairs** (844 pass/pass, 171 fail/fail, 15 disagreements). That is 39% of the accepted set: the sharded run was stopped once agreement had stabilized, at 98.9% over 458 pairs and 98.5% over 1,030, since this is a validation whose expected value is near 100% and whose residual uncertainty at that sample size cannot reach the headline. All 15 disagreements resolve as follows. Seven are our replica being *stricter*, which cannot inflate the finding; all seven are in the **norm** class. The other 8 are rows our replica accepts and their code rejects: 2 where their harness raised before any comparison happened (a bool-tensor subtraction and a Triton compile error) and 6 input-draw-dependent value mismatches, whose maximum differences run from 0.14 to 5.10 and which span five operator classes. The bound is therefore 8 rows of 1,030 in one direction against 7 in the other, a net of one row inside the sample. The two patches we needed were scoped to *loading*, not checking (device-index resolution and importing from a temporary file so that Triton traces the source), so the correctness logic under test is byte-for-byte theirs. Their code ran on 1,030 of the 2,638; the differential of Section 3.4 is computed with our replica over all of them, and what this check establishes is that the replica and their code agree 98.5% of the time wherever both ran.

(4) Stratified hand-audit. We hand-traced 31 disputed cases (accepted by the benchmark, rejected by us), stratified across every channel, and classified each from the gate semantics and the failure signature: **16 are genuinely broken**, **8 are real but tolerance-dependent** (the class a loose `allclose` hides, and the reason the differential cell exists), and **7 are reclassified as out of scope** and dropped from the strict floor. Of the 16, **11** fail on a formally tolerance-free channel and 2 more mint a NaN from a finite input, which no tolerance excuses even though a thresholded gate is what caught it; the remaining 3 are genuine on the evidence but rest on a threshold, and are excluded from the floor’s language. The sample was drawn before the applicability and artifact filters of Section 3.2, over a frame of 1,533 rather than the 1,487 the cell holds once those filters are applied, a 46-row gap, which is why one of the 31 is a row the audit itself goes on to exclude. The audit discards 7 of the 31 as out of scope, which is evidence that it does not rubber-stamp its own conclusion. Every identifier below resolves in the released per-row file. Two are confirmed failures with no tolerance argument available. Id 394 is a `matmul` that hands back the same output storage across calls: its recorded 4.04× speedup and its aliasing are the same fact, so the number that made it attractive is the defect. Id 28 is an `RMSNorm` that returns a NaN when fed a finite 10^6 input, a non-finite minted out of ordinary numbers. Both were accepted by the standard check at both its paper-era and its hardened tolerance. The third, id 518, is one of the seven we discard: its operator takes no floating-point tensor input for

the gates to perturb, so the verifier marks it not-applicable and it leaves the denominator rather than counting against it, and yet the external timing loop still recorded a 36× “speedup” for it. We report it as evidence about timing-first acceptance, not about silent wrongness, and it is counted as neither.

3.4 The differential

Running the same accepted kernels through KernelBench’s standard paper-era check (`allclose atol=rtol=10-2`, five random-input trials, fixed shapes) [8] produces the 2×2 contingency table of Figure 3. The benchmark accepts **93.7%** (2,472 of 2,638) of these kernels. The load-bearing cell, external PASS and our FAIL, is **1,487** (56.4% of the accepted set), of which 958 fail on a tolerance-free gate. Re-running the whole join moves that cell slightly, since both checks draw random inputs and 126 of these kernels are measurably nondeterministic, so we release two full runs of the same 2,638-row join rather than one: the second reads 1,482 with 954 tolerance-free, and 15 rather than 14 in the reverse cell. The cell is stable to about five rows, which is well inside the distance to any conclusion drawn from it. The reverse cell, external FAIL and our PASS, is only **14** (0.5%), so the two checks disagree almost entirely in one direction, which a uniformly stricter `allclose` would not produce. Under KernelBench’s *hardened* per-dtype variant (`fp32` tolerance 10^{-4}) the accept rate falls to 84.6%, but **1,263** kernels still pass their check and fail ours, so the finding survives the tightened check. The standard test certifies nearly 1,500 broken kernels as correct.

	Ours: PASS	Ours: FAIL
KernelBench: PASS	985	1,487 958 tolerance-free
KernelBench: FAIL	14	152

Figure 3: The differential, a 2×2 contingency table with each cell shaded by its count. KernelBench’s own paper-era check accepts 93.7% of these kernels; the load-bearing cell (accepted by the benchmark, rejected by us) holds 1,487 kernels, 958 of them on a tolerance-free gate, while only 14 go the other way. The near-unidirectional disagreement is the signature of a systematic blind spot in the acceptance signal, not of two checks tuned to different strictness.

3.5 Robustness: a second stack and a second corpus

The finding reproduces on a different software stack: a 300-kernel sample re-audited on torch 2.11.0+cu128, against the audit’s 2.12, returns **68.6%** over the 245 of those rows that survive the same accepted-only and artifact filters, on the same B200 silicon. It runs above the 62.1% headline. Composition does not explain the gap: the high-rate reduction class is *under*-weighted in the re-audit (26.5% of rows against 28.7%), and standardising the re-audit to the main corpus’s class mix moves it further up, to 68.9%. The per-class rates are simply higher on the older stack in six of the seven classes, reduction at 84.6% against 71.8% and softmax at 59.4% against 47.4%. We read that as the finding being, if anything, understated by the newer toolchain. The pre-registered kill criterion was a re-audit rate below 5%; the observed 68.6% clears it by more than thirteen times. A second corpus

of *native CUDA* kernels, the Sakana AI CUDA Engineer archive [9], shows a related but *weaker* pattern. The selection rule is the archive’s Level-1 split, restricted to rows the system’s own `Correct` boolean marks correct and that expose the `KernelBench Model/get_inputs` convention our harness needs; the first 214 such rows were audited, one timed out in the sandbox, leaving 213. Its raw 100% fail rate is precision-dominated, because the kernels are `fp32`-only by design, so we do not report it as a headline figure. The environment-robust residual is **20.2%** (43 of 213), driven by `CMP-03` shape rigidity (37), `EXC-02` subnormal handling (9), `ORD-02` nondeterminism (4), and `CMP-01` value (1). These per-gate counts total 51 across 43 distinct kernels, because 6 kernels fail more than one non-precision gate, one of them failing four. There is no mass silent-wrongness (`CMP-01` 1/213; median maximum difference 0.001). This is the same rigor gap recurring in native CUDA along a complementary and weaker axis of precision and shape rigidity, rather than as a second 62%.

4 A Native `tcgen05` Backward for the GDN Family

The second contribution is a piece of systems software that the verifier and a double-precision oracle jointly certify: a hand-written native Blackwell `tcgen05` tensor-memory training backward for the gated-linear-recurrence family.

4.1 One recurrence, five models

A Mamba-style selective scan keeps a vector state h and updates $h_t = \bar{a}_t \odot h_{t-1} + \bar{b}_t \odot u_t$, $y_t = C_t^\top h_t + D \odot u_t$, where the per-step decay $\bar{a}_t$ and write $\bar{b}_t$ are recomputed from the input at every step. That input dependence, known as selectivity, is what makes the model expressive and what forbids precomputing the scan as one matrix multiply. The models we target use a matrix state S of shape $d_k \times d_v$, and the stronger gated DeltaNet (GDN) update. Writing q_t, k_t for queries and keys, v_t for values, and three gates g_t (per-channel log-decay), b_t (erase), and w_t (write),

$$S_t = (I - k_t(b_t \odot k_t)^\top) \text{Diag}(e^{g_t}) S_{t-1} + k_t(w_t \odot v_t)^\top, \quad o_t = S_t^\top q_t. \quad (1)$$

Equation (1) combines channel-wise decay, the delta rule (the model subtracts what S already predicts for a key before writing, so it stores the correction rather than the raw value), a rank-one write, and a read with q_t . The delta rule couples every step to the entire state, and the backward must unwind that coupling in reverse.

Equation (1) is a superset. Fixing its gates recovers named models (Table 2), so one backward differentiating (1) once produces the training gradients for all five. Each member is re-derived independently from its own definition and checked against *its own* reference, not by setting knobs on the GDN path. Two reductions are exact enough to serve as built-in tests: $b=w=\beta$ reduces (1) to the KDA equation, pinned in `fp64` at 10^{-12} rather than bitwise because the two paths contract in different orders, and $b=0$ makes an entire triangular-solve stage vanish ($M=0 \Rightarrow T=I$), which is pinned bitwise.

Table 2: One recurrence, five models. Each named model is a gate setting of the general recurrence, so a single hand-written backward trains them all, and each is verified against its own independent reference.

Model	Recurrence $S_t = \dots$	gate setting
LA (linear attention)	$S_{t-1} + k_t v_t^\top$	$g=b=0, w=1$
GLA (gated linear attention)	$\text{Diag}(e^{g_t}) S_{t-1} + k_t v_t^\top$	$b=0, w=1$
SSD / Mamba-2 (scalar decay)	$e^{g_t} S_{t-1} + k_t v_t^\top$	$b=0, w=1, \text{ scalar } g$
KDA (Kimi Delta Attention)	$(I - \beta_t k_t k_t^\top) \text{Diag}(e^{g_t}) S_{t-1} + \beta_t k_t v_t^\top$	$b=w=\beta_t$
GDN (gated DeltaNet)	the full recurrence (1)	per-channel b, w, g

4.2 The two hard kernels

The scan is sequential; the GPU wants parallelism. The standard resolution is *chunking*: split the sequence into chunks of 64 steps, do the parallelizable work inside all chunks at once, then run a short sequential carry between chunks. The backward runs this in reverse, and two stages are genuinely hard.

- **K#1, the reverse inter-chunk state scan.** A reverse-time walk accumulating the state gradient dS (shape $d_k \times d_v$, fp32) backward across chunks, with a rank-one correction at each step from the delta rule. This is the sequential, carried stage, and it is the exact piece that open libraries still run on a Triton fallback. Making it native `tcgen05` is the sharpest single element of the contribution: one tensor-memory reservation held across the entire unrolled reverse scan, both matrix-multiply accumulators addressed at fixed column offsets inside it, and a single release at the end. The dS carry itself is fp32 throughout, updated by the kernel’s own threads between chunks rather than kept in tensor memory, which is reserved for the accumulators.
- **K#2, the WY / triangular-inverse VJP.** Inside each chunk the delta rule requires a small triangular solve $T = (I + M)^{-1}$, and its vector-Jacobian product is the numerically nastiest piece. The forward already computed T , so the backward reuses it in four triangular products, one for each of the two applies and two for the $T^\top(dT) T^\top$ sandwich that differentiates the solve, and never re-inverts.

Everything else in the backward is genuine work, but it is supporting glue. The design keeps K#1 and K#2 as native kernels while the glue is torch, and the glue is progressively fused in.

4.3 Scope of the novelty claim

As surveyed in July 2026, against pinned upstream revisions recorded with the evidence: NVIDIA ships an SSD forward but no backward in either of its two stacks; cuLA has a KDA backward that is a *hybrid* (one native CuTe kernel for the WY stage, but the reverse-state K#1 stage still in f1a Triton); FlashKDA is forward and inference only; and `tilelang` has GDN/KDA backward examples with no `tcgen05`/TMEM in them. The contribution is therefore two firsts: a native Blackwell `tcgen05`/TMEM *training backward* for the GDN family, and a native `tcgen05` *reverse-state* backward, the stage others still run in f1a Triton. We do *not* claim the first native backward kernel for the family (cuLA’s WY predates ours), the first non-Triton GDN/KDA backward (`tilelang` exists), or bit-for-bit parity (our parity is fp64-tight, not bitwise).

4.4 Clearing the tensor-memory constraint

The first attempt to run two or more matrix multiplies in one kernel produced illegal machine code or a deadlock, the same failure class behind #904. The root cause, established by reading NVIDIA’s own `mamba2_ssd.py` in the same pinned toolchain, was a lifecycle error: the kernel reserved the full 512-column TMEM budget and released it *per matrix multiply*, which the hardware forbids. NVIDIA’s kernel instead runs four `tcgen05` MMAs, including the cross-chunk recurrence, with a *single* reservation, fixed per-accumulator column offsets, and *one* release at the end. Porting that lifecycle, offset-partitioned accumulators with alloc-once and relinquish-once, cleared the blocker. What it unblocked is the contribution itself: the native reverse-state scan and WY-VJP for the GDN family, hand-written on the (128, 64, 128) `tcgen05` tile with TMA data movement and async pipelines, the backward stages that open implementations otherwise keep in f1a Triton.

4.5 Verification and the oracle chain

Correctness is anchored by a layered ground-truth ladder, each rung checked against the one below: a token-serial fp64 oracle, a chunkwise reference, an fp64 torch assembly of the K#1/K#2 references with glue, and finally the `tcgen05` kernels on the B200. Applied to the native backward, the ladder gives the following:

- **Double-precision spec pins** on the reverse-state-scan tile and the WY-VJP tile, machine-exact against the closed-form spec: the committed fp64 cross-check runs from 5.9×10^{-17} to 1.7×10^{-16} across shapes, against a 10^{-9} bound.
- **The full assembled backward against an independent fp64 oracle:** worst relative error 3.29×10^{-3} (scalar) and 3.31×10^{-3} (channel-wise), both under the 5×10^{-3} acceptance bound, and bit-for-bit deterministic across runs. One configuration sits outside that bound: at the widest $d_v=128$ shape the assembled pipeline reaches 5.21×10^{-3} , a 4% overrun. The same run records that figure on both the closed and the save-forward arm, so it belongs to the shape

and to the assembly, not to the kernel, which measures 5.5×10^{-4} against the same fp64 oracle in its shipped gate configuration and 6.9×10^{-4} under a drift-stressed one, and the acceptance claim is scoped accordingly.

- **It trains.** The native backward runs real 300-step training loops with zero numerical blow-ups, once a masked-exp2 fix corrected an exponent overflow that had produced a NaN. Every family member trains through the native path with the reference fallback disabled, that is, on the real kernels: the four reductions (LA, GLA, SSD, KDA) and the full GDN recurrence, which the backward differentiates directly. Four of the five clear the training gate in 300 steps at $L=256$, converging to between 1.5×10^{-2} (GLA) and 9.5×10^{-5} (the full GDN recurrence on the shipped fused default; 8.9×10^{-5} on the further-fused arm). LA is the exception and we state it fully: it fails the gate at $L=256$, where 600 steps move the loss only from 3.61 to 3.07, and clears it only at $L=64$ over 4000 steps, where it still converges no further than 1.53. That is a property of the ungated $g=b=0$ reduction’s conditioning, not of a kernel disagreeing with its reference, but it is a weaker result than the other four and we do not average it away. Fresh-input graph replay is bit-exact against eager execution.
- **Positive control.** The committed test suite runs the full twelve-gate battery of Section 3 on the native backward, supplying resource metadata so RES-02 is applicable and per-gradient views so CMP-02 has a graph to differentiate, and it passes all twelve. In the positive control of Section 3.3 it is judged instead by the same ten-gate audit subset as the foreign corpus, and is clean on every tolerance-free channel there.

Verification envelope. The committed CPU test suite validates the mathematical scaffolding, the references, the assembly, and the fp64 spec pins. The tcgen05 kernels themselves are exercised only on the B200, over a narrow shape envelope ($d_k=128$, $\text{chunk}=64$, $d_v \in \{64, 128\}$) at a loose fp16 tolerance. This is a real limitation and we return to it in Section 5.

4.6 Speed

The native GDN backward is *slower* than the f1a Triton library [19], by roughly $8\times$ at $L=512$ rising to about $78\times$ at $L=2048$; we do not claim parity. The gap is *structural*: f1a sits near a 0.9 ms latency floor because it reuses the delta-rule inverse saved in the forward pass, while our reverse-state scan is inherently sequential and most of our runtime is spent in the surrounding fp32 glue rather than in the tensor-core kernels. The speedups we report are over our *own* earlier pipeline, not f1a: a channel-wise fusion campaign cut the captured backward $2.75\times$ (from 52.9 to 19.2 ms), and a tensor-memory tiling optimization cut the $d_v=128$ save-forward variant $2.98\times$ (from 24.50 to 8.23 ms). The contribution is that the kernel is native, general across the family, and verified; speed is future work.

4.7 The bridge: the audit’s failure modes are the gates that judged our own kernel

The verifier’s two uses, the outward audit of Section 3 and the inward acceptance gate of this section, are joined not only by shared code but by shared data. The defects that most often sink foreign kernels are, gate for gate, the defects our own kernels were at risk of and in one case actually had, and the hardware constraint the native backward had to clear is the same one the verifier’s RES-02 gate is built to detect. Table 3 aligns the two. We read the alignment as the strongest available evidence that the instrument’s outward and inward uses are one argument: a checker that flattered its author would neither have caught the author’s own bugs nor shared a failure surface with the corpus it indicts.

Three alignments are substantive. The tensor-memory constraint is the sharpest: the lifecycle error described in Section 4.4 is the exact failure class behind the open #904 bug, and it is the constraint the verifier’s RES-02 gate exists to detect. Because the audit corpus is forward-only, RES-02 contributes no foreign failure count, so this row bridges through the taxonomy and through our own reproduction of #904, with no measured rate behind it.

The second alignment is shape rigidity. CMP-03, shape breakage, fails 18.1% of applicable foreign kernels, and it is the gate that surfaced the C5 channel-inference defect described in Section 3.3. That the same gate which rejects foreign kernels also flagged our own, and that the fix changed the shipped operators rather than only the verdict, is the concrete form of the fairness argument. It is also the one row where we changed the reference, stated in full in Section 3.3.

Table 3: The bridge. Each row aligns a failure mode the audit finds in foreign kernels with the same gate acting on our own kernel. The first three rows are the substantive alignments (a caught bug or a shared hardware constraint); the last two are pass-by-design robustness. All figures are drawn from Section 3 and Section 4; no new numbers are introduced.

Gate (foreign fail rate)	Failure mechanism	Where our kernel was at risk	How the gate resolved it	Bridge strength
RES-02 (idle in audit; = #904 class)	Requests 544 TMEM columns vs. the 512 budget, so the kernel yields illegal code, a deadlock, or a slow fallback	First multi-GEMM attempt hit the same failure class (Section 4.4): reserved 512 and released per-MMA, which the hardware forbids	RES-02 detects #904; cleared with alloc-once / relinquish-once and offset-partitioned accumulators	Strong (shared hardware constraint)
CMP-03 shape breakage (18.1%)	Correct at the tested shape, wrong at another	C5 fused_block_forward inferred channel dim from input without checking conv-weight channels (Section 3.3)	Positive control flagged it; guard added to both ops and the reference (commit 7f46226), so the degenerate variant is reference-inapplicable as on six of the seven control kernels; control then 7/7	Strong (real bug, code changed on both sides)
EXC-01 non-finite non-propagation (34.2%, modal)	Silently returns a finite number where the reference is NaN/Inf	Two NaN-generating bugs: masked-exp2 exponent overflow (Section 4.5); A_log drift, decay $e^{\delta A} > 1$ state blow-up (Appendix C.3)	Caught because our kernel propagated the NaN instead of swallowing it, the inverse of the foreign defect	Strong (propagation makes bugs catchable)
PRC-02 dropped fp32 accumulator (13.8%)	Reduction accumulates in fp16, losing precision	dS state gradient is carried in fp32 across the whole reverse-state loop by design, never demoted to the fp16 operand dtype	Passes by construction; fp64 oracle chain would expose any lapse	Supporting (pass-by-design)
ORD-02 nondeterminism (4.9%)	Different output across runs / buffer aliasing	n/a (not at risk)	Bit-for-bit deterministic across runs; graph replay bit-exact vs. eager (Section 4.5)	Supporting (pass-by-design)

The third alignment is the modal foreign defect, and it carries the sharpest form of the thesis. EXC-01, non-finite non-propagation, is the single most common failure in accepted foreign kernels (34.2%): a kernel silently returns an ordinary number where the reference returns a NaN or an infinity. Our own kernel generated non-finite values twice during development, an exponent overflow fixed by a masked-exp2 (Section 4.5), and a drifted A_log parametrization that made the state decay exceed one and blow up over long sequences, fixed by the standard $A = -\exp(A_log)$ form (Appendix C.3). The instructive point is the asymmetry: the foreign defect is swallowing a non-finite, whereas our kernels propagated theirs, and it was precisely that propagation which made the errors visible at step 250 and during training. Non-finite propagation is the property that makes a kernel’s own failures catchable. A kernel that had silently substituted a finite value, the foreign failure mode, would have hidden the same bugs from us.

Two further gates, PRC-02 (fp32 accumulation) and ORD-02 (determinism), our kernel passes by construction rather than by catching a bug: the reverse-state gradient dS is carried in fp32 across the whole loop while the tensor-core operands around it are fp16, and the backward is bit-for-bit deterministic across runs with graph replay bit-exact against eager execution (Section 4.5). We list them in Table 3 for completeness and mark them as pass-by-design, because a positive control that passed only by design would be weaker evidence than one that caught something; the substantive alignments are the first three.

5 Discussion

Why a tolerance-free floor is the right lens. Every prior treatment of generated-kernel correctness ultimately parameterizes acceptance by a tolerance, and a debate over the number follows. The tolerance-free floor sidesteps that debate. A kernel that returns a finite value where the reference returns a NaN, or a different answer on each run, or that aliases a shared buffer, crashes, or does not run at all, is wrong under any tolerance, and 39.5% of an accepted corpus fails in exactly this way; adding shape rejection raises it to 41.1%. We therefore lead with the floor as the conservative claim and treat the 62.1% as the wider, tolerance-dependent rate above it. The two are computed over

slightly different channel sets, so the floor is a subset of the violation rate only once aliasing is folded into ORD-02 as Table 1 defines it, which moves the wider figure to 62.5%; we report the smaller number. The modal defect is instructive: EXC-01 non-finite non-propagation is a silent substitution of an ordinary number for a NaN or an infinity, which converts a detectable training-time error into undetectable data corruption. The most common failure in accepted kernels is of this kind, which makes the acceptance gap qualitative: a slightly-too-loose threshold does not produce it.

The differential is directional, and that is the point. A stricter checker that simply flagged more kernels would disagree with the loose check in both directions. Ours does not: the reverse cell, kernels the benchmark rejects but we accept, is 14 of 2,638 (0.5%). The disagreement is almost entirely one-directional, 1,487 to 14, which is the signature of a systematic blind spot in the acceptance signal rather than of two checks calibrated to different strictness. Combined with the 98.5% agreement against KernelBench’s own correctness code, this supports a specific claim: the gap is a property of *what the standard check probes*, not of a disagreement about how close is close enough.

On the fairness of the referee. The strongest threat to any audit like this is that the verifier is tuned to fail foreign code. We designed the four defenses to be falsifiable rather than rhetorical. The positive control could have failed and did surface a real gap in our own code, which we fixed in the shipped operators, down to the reference change the same commit carries. The calibration sweep could have shown a gate firing inside the correct-kernel noise band and instead shows clean separation on every gate but the thin one. The external-agreement check could have diverged from KernelBench’s code and instead matches it 98.5% of the time, with the 15 disagreements bounded and enumerated in Section 3.3. The hand-audit could have confirmed all 31 disputed rows and instead discards 7 as out of scope. We read the convergence of four independent and separately falsifiable defenses as the reason the finding should be believed.

A slow certified kernel still supports the thesis. The native backward is slower than `fla`, and the gap is structural. The contribution of the native kernel is that it is native, general across the family, and verified, closing a real implementation gap (the reverse-state stage open implementations run on a fallback) and getting the tensor-memory lifecycle right where the official kernel does not. That the same verifier which exposes foreign kernels also measures our own speed deficit precisely, with the structural mechanism named and the one numerical exception stated, shows the methodology working as intended.

Threats to validity. We note four. *Corpus dependence*: the headline rests on one primary corpus, a dependence mitigated by the cross-stack reconfirmation (68.6% on torch 2.1.1) and by the second, native-CUDA corpus (20.2% residual), which together show that the effect is neither toolchain-bound nor Triton-specific, though it is weaker on CUDA. *Gate coverage*: on a forward-only corpus, two of twelve gates are idle, so the audit rests on ten and effectively seven load-bearing gates. Both run against our own kernels in the committed test suite, which is what establishes that they fire, but nothing in this corpus exercises them, and a corpus carrying autograd graphs would exercise CMP-02 directly. *Reference correctness*: the audit is only as good as its references, which is why the red team of nineteen cheats and the positive control on known-good kernels both run, and why the referee never grades against a reduced-precision computation of the answer. *Native-kernel envelope*: the `tcgen05` kernels are verified on a narrow shape envelope at a loose `fp16` tolerance on a single GPU generation, so their correctness claim is scoped to that envelope and not asserted beyond it.

Limitations. Beyond the envelope above, the native backward is kernel-accelerated rather than fully native: stage-B, the normalization gradient, packing, and masks remain in torch and are progressively fused in. The one accuracy exception, the $d_v=128$ shape at 5.21×10^{-3} , is scoped in Section 4.5.

Implications and future work. The practical implication is that acceptance criteria for kernel-generation benchmarks understate the correctness gap, and that a small set of tolerance-free contracts (non-finite propagation, determinism, shape polymorphism) would close most of it at modest cost; we suggest such contracts as a benchmark standard. On the systems side, the structural speed gap points to a specific lever (a `tl.dot` path on a post-#9093 Triton, and reducing the surrounding `fp32` glue) rather than a general redesign.

6 Conclusion

Every reported result in GPU-kernel generation rests on a claim of correctness, and that claim rests on a test too weak to carry it. We built the instrument that tests it properly, a contract-grade verifier of twelve adversarial gates, several of them tolerance-free, and we aimed it in two directions. Aimed outward at a corpus of machine-generated kernels that a public system’s own harness had already accepted as correct, it found 62.1% carrying a contract violation and 39.5% broken in a way no tolerance argument can excuse. Aimed inward, the same battery judged a kernel of our own: the first native Blackwell `tcgen05` training backward for the gated-linear-recurrence family, including the reverse-state stage the field still runs on a fallback. That kernel’s correctness was established independently of the verifier, against a double-precision oracle, machine-exact on the core tiles and 3.3×10^{-3} end to end with one $d_v=128$ configuration at 5.21×10^{-3} , so its passing is a genuine positive control rather than a self-assessment. The same instrument that clears it is what lets us state its speed deficit against the frontier Triton library precisely, and that deficit is a structural negative. One instrument, two directions, and the same standard in both: the defects the verifier found in our own work are what make the defects it found in everyone else’s worth believing. The correctness behind the field’s reported progress is far weaker than its numbers suggest, and the same twelve contracts that expose the gap are enough to begin closing it.

A Mathematical Details

Every formula here is traceable to the Mamba-3 paper [3] and the official `state-spaces/mamba` source.

Mamba-3 SISO forward. The continuous-time complex state-space model is $\dot{h} = \text{Diag}(A + i\theta) h + (B + i\hat{B}) x$, $y = \text{Re}((C + i\hat{C})^\top h)$. After exponential-Euler discretization it becomes a real SSM

$$h_t = e^{\Delta_t A_t} R_t h_{t-1} + \Delta_t B_t x_t, \quad y_t = C_t^\top h_t,$$

where $B_t = [B_t; \hat{B}_t] \in \mathbb{R}^N$ and $C_t = [C_t; -\hat{C}_t] \in \mathbb{R}^N$ stack the real and imaginary halves into one $N=d_{\text{state}}$ vector, and R_t is the block-diagonal rotation built from $\begin{bmatrix} \cos \Delta_t \theta_{t,k} & -\sin \Delta_t \theta_{t,k} \\ \sin \Delta_t \theta_{t,k} & \cos \Delta_t \theta_{t,k} \end{bmatrix}$ over dimension pairs k . The RoPE angle is data-dependent and cumulative, $\theta_t = \sum_{s \leq t} \tanh(\text{angle}_s) \Delta_s \pi \pmod{2\pi}$, so the rotation at step t depends on the whole history and needs its own causal cumsum before the scan. We write R_t as acting on h_{t-1} ; our reference implements the equivalent factorization that applies the rotation to B_t and C_t instead, which is cheaper and gives the same y . The repository also carries the trapezoidal discretization of the same model, $\lambda = \sigma(\text{trap})$, which is bit-identical to the exponential-Euler form at $\lambda=1$; the six kernels of Appendix C.1 are the $\lambda=1$ path.

Mamba-3 MIMO. With input rank j and output rank i , the state decomposes into R independent SISO scans with no cross-terms, read out over R^2 rank pairings,

$$h_t^{(j)} = \alpha_t h_{t-1}^{(j)} + \Delta_t B_t^{(j)} x_t^{(j)}, \quad h_t = \sum_{j=0}^{R-1} h_t^{(j)}, \quad y_t^{(i)} = (C_t^{(i)})^\top h_t, \quad y_t = \sum_{i=0}^{R-1} y_t^{(i)} \phi_i,$$

where $x_t^{(j)} = x_t \odot \psi_j$ is applied before the state update and ϕ_i after the readout, both learnable and data-independent. The backward accumulates dS over all output ranks first, then contracts per input rank; the two sums decouple.

The GDN backward’s two hard stages. For the general recurrence (1), the training backward reduces to the reverse-state scan $dS_t = \text{Diag}(e^{g_t}) dS_{t+1} + (\text{rank-one delta correction})$, walked backward, and the WY / triangular-inverse VJP, which differentiates the chunk-local solve $T = (I + M)^{-1}$ by reusing the forward’s T across four triangular products rather than re-inverting. The channel-wise path folds the per-channel decay $\text{Diag}(e^{g_t})$ inside those contractions; the scalar path ($b=w=\beta$) recovers only channel sums of some gradients, which is why it serves as a reduced de-risking check while the channel-wise path carries the headline result.

B Detailed Numerical Results

This appendix tabulates the exact numbers behind Figures 1 and 2.

Table 4: Per-gate results among the 2,638 accepted kernels (Figure 1, bottom panel). Count is the number of kernels that failed the gate; rate is that count over the kernels where the gate *applied*, excluding both rows the reference could not run and rows the gate errored on, so the two columns are not directly divisible and the per-gate rates are not additive to the corpus-level 39.5% floor. PRC-01’s rate is over its half-precision rows alone, since an fp32 failure is charged to CMP-01 instead. Tolerance-free gates are bold; EXC-02 is tolerance-free but is deliberately held out of the floor (Appendix B.2). CMP-02 and RES-02 have no applicable rows in this forward-only corpus, as Section 3.1 explains, and run against our own kernels in the committed test suite rather than here.

Gate	Fail count	Fail rate
EXC-01 non-finite propagation (NaN/Inf swallowed)	868	34.2%
CMP-01 value on varied input	608	23.6%
CMP-03 breaks on a new shape	420	18.1%
PRC-02 missing full-precision accumulator	352	13.8%
ORD-01 reduction-order tolerance	351	13.7%
PRC-01 precision regimes	287	11.3%
EXC-02 subnormal handling	238	9.3%
ORD-03 adversarial reduction	136	5.3%
ORD-02 nondeterministic output	126	4.9%
RES-01 device residency	45	1.8%

Table 5: Threshold calibration (Figure 2). Each gate’s threshold sits between the noise a correct kernel emits and the error a wrong one emits. ORD-03’s thin upper margin (1.2×) is thin; it is the deliberately-relaxed gate. Lower noise floor and higher real-error onset are better; a wide margin on both sides is the goal. Margins are computed from unrounded measurements, while the noise-floor, threshold, and onset columns are displayed to two significant figures, so the printed ratios are not exactly reproducible from the printed values; ORD-03’s upper margin, for instance, is 1.243× unrounded.

Gate	Noise floor	Threshold	Real-error onset	Margin below	Margin above
CMP-01	2.9e−7	4.2e−5	2.4e−3	147×	57×
CMP-03	2.9e−7	4.2e−5	2.4e−3	147×	57×
ORD-01	2.9e−7	3.1e−5	2.4e−3	110×	76×
ORD-03	2.9e−7	1.0e−3	1.3e−3	3591×	1.2×
PRC-02	passes ∀N	2e−2	3.5e−2 (N=2048)	n/a	crosses N≥2048

B.1 Corpus selection and exclusions

Section 3.2 states two steps without operationalizing them. Neither involves a judgement call at audit time: the classifier is a pure function of the public corpus row, and the artifact filter is a pure function of the recorded run status.

Operator class. Each trajectory’s *reference* PyTorch source is matched against an ordered list of regular expressions and takes the class of the first pattern that matches: attention (scaled_dot_product_attention, MultiheadAttention, attention; case-insensitive), then scan (cumsum, cumprod, cummax, cummin, selective_scan, scan; the single-word patterns anchored at word boundaries), then matmul (matmul, bmm, mm, addmm, einsum, nn.Linear, infix @), then conv (Conv[123]d, conv[123]d, conv_transpose), then norm (LayerNorm, RMSNorm, GroupNorm, BatchNorm, InstanceNorm, layer_norm, rms_norm), then softmax (softmax, log_softmax; case-insensitive), then reduction (.sum(), .mean(), .max(), .min(), .prod(), logsumexp, amax, amin, .var(), .std(), and the torch.-qualified forms of sum, mean, max, min), then elementwise, then *other*. The seven SSM-adjacent classes are the first seven; elementwise and other are not audited.

Why the rule is weak enough to be safe. A first-match-wins cascade is a coarse instrument, and we use it only where coarseness cannot propagate. Three properties bound what it can affect.

First, the *audited population is invariant to the order of the seven patterns*. Membership is determined by whether any of the seven matches, and a union does not depend on the order of its terms. Ordering decides only which of the seven label a row carries, so a normalization written on top of `nn.Linear` is labelled `matmul` rather than `norm`, and that reassignment moves a row between two slices we report separately without adding or removing a single kernel from the denominator. The only boundary that can change the population is the one between the seventh pattern and `elementwise`, and that boundary is inclusive in the conservative direction: a reference containing any reduction call is audited even when the operator is otherwise `elementwise`, so the rule over-admits rather than curates.

Second, *no verdict depends on the label*. Gates, references, tolerances, and applicability are all derived from the reference operator’s runtime behavior, its dtypes and its shapes. The class string is never read by the verifier. It exists to define which slice of the corpus we audited and to let a reader cut the result by operator family.

Third, and decisively, *the result does not rest on any class*. Table 7 gives the tolerance-free floor for each of the seven and for the corpus with each one removed. Every class independently exceeds 25%, and removing any single class leaves the floor between 31.6% and 41.9% against a reported 39.5%. Reduction is the class the floor is most sensitive to, and dropping it entirely still leaves 31.6%. A misclassification severe enough to matter would have to move a large population between the audited seven and `elementwise`, in the direction that removes kernels, and would still have to survive this spread.

The three filters. Table 6 gives the counts. A row is a *toolchain or compile artifact* when its status is a sandbox unpickling error, or when the candidate failed to run natively with an error beginning `CompilationError`, `UnsupportedLanguageConstruct`, `OutOfResources`, or `PTXASError`. These are properties of our sandbox and of the pinned toolchain rather than of the kernel’s numerics, so charging them to the candidate would inflate the finding. Excluding them is the conservative direction: every one of the 281 is dropped from the denominator rather than counted as a failure.

Table 6: From the audited operator classes to the headline denominator. Filters are applied in the order shown.

Step	Dropped	Remaining
SSM-adjacent operator classes		3,134
Accepted by the source harness (<code>final_speedup > 0</code>)	164	2,970
Reference exposes a floating-point tensor input to perturb (else not-applicable)	51	2,919
Not a toolchain or compile artifact	281	2,638

Table 7: The tolerance-free floor is not carried by any one operator class. *Own rate* is the floor within that class alone; *floor without it* is the corpus-level floor recomputed with the class removed entirely, against a reported 39.5%. Attention is listed for completeness; at $n=8$ its own rate carries no weight, and removing it moves the corpus figure by 0.04 points.

Class	Kernels	Own rate	Floor without it
reduction	758	59.1%	31.6%
matmul	751	34.2%	41.7%
softmax	390	26.2%	41.9%
norm	352	34.7%	40.3%
conv	239	31.0%	40.4%
scan	140	27.1%	40.2%
attention	8	25.0%	39.6%
All	2,638	39.5%	

B.2 Composition of the tolerance-free floor

The 39.5% floor is the union of five channels, each of which admits no tolerance argument: EXC-01 (a non-finite is swallowed), ORD-02 (output differs across repeats), output aliasing into a shared buffer, a CUDA error raised during gating, and a non-artifact failure to run at all. Their individual counts over the 2,638 are 868, 126, 25, 16, and 62; they sum to 1,097 and their union is 1,043, so the channels overlap on 54 kernels.

EXC-02 is tolerance-free by the same standard and is *not* in this union. Subnormal flush-to-zero is the one member of the tolerance-free group where a reader can reasonably answer "that is a convention, not a defect", since production kernels enable FTZ deliberately. We hold it out of the floor for that reason and report its effect: including EXC-02 raises the floor to 1,154 (43.7%). The reported 39.5% is therefore the conservative reading of our own gate set, not the maximal one.

C Supporting Results

Around the two headlines sit five supporting results, each reported with the number behind it.

C.1 Six contract-verified Mamba-3 kernels

We hand-wrote six Triton kernels for the Mamba-3 model (Table 8), with no `tl.dot` anywhere. Because they never touch the broken compiler pass, they compile and run across the `num_warps` 2/4/8 sweep on Blackwell, which the committed bench artifacts record for five of the six, where the official kernel loses every `num_warps` ≥ 4 configuration to the TMEM budget. This is an availability advantage, categorical and independent of any latency ratio. The six serve double duty: they are the forward and backward pieces used by the applied demonstration, and, because they were verified against hand-derived references *first*, they are the known-correct kernels against which the verifier's tolerance model was calibrated before it was ever aimed at foreign code. We keep the two roles apart deliberately. These six fixed the *thresholds*; the positive control of Section 3.3 tests the *verdicts*, and the native GDN backward of Section 4 played no part in calibration, which makes it the one control among the seven that the thresholds were never fitted to. Four of the six drove three calibration findings that were folded back into the verifier: the scale-aware `ato1` model from C1, PRC-02 `fp16-carry` separation from C2, C3, and C5, and EXC-02 subnormal-flush handling for affine ops from C5. The CMP-03 shape-rigidity convention sits on top of these as a cross-kernel design rule rather than a finding any one kernel surfaced. All six pass every applicable gate on B200, and the committed record of the full GPU test suite is **1,191** passed, 3 skipped, 0 failed.

Table 8: The six hand-written, contract-verified Mamba-3 Triton kernels, separate from the native GDN backward of Section 4.

#	Kernel	What it is
C1	<code>forward_chunked_scan</code>	the forward scan; surfaced the $\varepsilon\sqrt{N}$ -scale calibration model the verifier uses
C2	<code>backward_selective_scan</code>	the training backward that avoids the #904 trap and compiles where the official one fails
C3	<code>mimo_backward</code>	the multi-input/output backward; the official MIMO backward is TileLang, and we make no priority claim for this one
C4	<code>complex_scan_rope</code>	a fused rotation-and-scan kernel in the real <code>cos/sin</code> form of the complex recurrence (data-dependent RoPE and decay in one pass)
C5	<code>fused_block_forward</code>	<code>conv1d</code> , <code>SiLU</code> , selective scan, and <code>RMSNorm</code> fused into one kernel
C6	<code>fused_block_backward</code>	the training-bottleneck backward for that fused block

C.2 A faithful reproduction of #904

We reproduced #904 on a B200 (torch 2.12 / triton 3.7). A batch-2 / $L=2048$ run and a batch-4 / $L=4096$ run both trip Required: 544, Hardware limit: 512 together with an out-of-resource error,

taking 18 autotune configurations out and leaving `num_warps=2` as the only survivor, which is the forced fallback the issue reports. The other half of the upstream report did not reproduce, and the negative is the more useful half. No `ptxas C7907` string appears anywhere in the log: on triton 3.7.0 the failure surfaces purely as an `OutOfResources` exception, so a detector keyed to the C7907 signature alone would have missed the live bug, whereas a resource contract stated over the TMEM budget catches it either way. That is the case for RES-02 in one sentence. We reproduced the tensor-memory half, our kernels survive it, and RES-02 detects it; we did not fix it upstream. This is the failure class that motivates both Strategy A (avoid `t1.dot`, the six kernels above) and Strategy B (master `tcgen05`, the native backward).

C.3 An applied demonstration: a 1.1B-parameter model

To show the kernels train a real model end-to-end, we trained a 1.10B-parameter selective-scan SISO model ($d_{\text{model}}=4096$, 32 layers, $d_{\text{state}}=64$, fp32) as a 12-lead physiological-signal classifier on the public PTB-XL dataset [15] (five diagnostic superclasses, multi-label), with data-parallel training across eight Blackwell GPUs and the forward pass running through our C5 fused kernel. It reaches a macro-AUC of **0.880** from random initialization with zero NaN (0.820 at step 500, then 0.859, 0.871, and a peak of 0.880 at step 2000; stopped at 2500 as validation loss rose, the overfitting onset). Two fixes unblocked training: the standard $A = -\exp(A_{\log})$ parametrization (the stored $\log A$ had drifted positive, so $\bar{a}=e^{\delta A} > 1$ blew up the state over $L=1000$ and produced a NaN near step 250; the official form makes A strictly negative and guarantees contraction), and moving the evaluation `all_reduce` onto the NCCL device. The block is the plain selective scan, that is, the Mamba-1 / S6 recurrence, which is Equation (1)’s SISO cousin with the rotation disabled; the Mamba-3-specific kernels are C3 and C4, and this model trains through neither. What it exercises is the C5 fused-block path. This is a demonstration that the stack trains a real model, not a domain result: 0.880 sits below the published state of the art for this dataset, which is a matter of training recipe (no pretraining, a mean-pool head, fp32-only, a fixed learning rate, and the SISO rather than MIMO block), not kernel correctness.

C.4 Reinforcement learning: an autotuner win and a discovery negative

Given a referee that scores any kernel objectively, a natural question is whether a code-writing policy in a loop can *discover* fast, correct kernels. We built the apparatus from scratch, GRPO [12] with a reward ladder that pays a speed bonus only after every contract passes (no compile $\rightarrow$ 0.0; compiles but fails the gates $\rightarrow$ 0.1; passes all gates $\rightarrow$ an anchor of 1.0 shifted by the clamped log of the speedup, so a correct kernel scores in $[0.5, 4.0]$ however slow or fast it is). An opt-in partial-credit mode gives a multi-gradient op that fails some views $0.1 + 0.35$ times the fraction it passed, capped at 0.45 and off by default. Every rung is below the correct kernel’s 0.5 floor, so a fast-but-wrong kernel can never out-score a correct one. The result is a clean split. **config-RL** (the policy emits only correctness-invariant launch knobs) is a modest win: it beats our shipped default by $1.167\times$ at long sequences and selects the right shape-to-mode boundary. But this is autotuning, not discovery, and the shape-gated selector we already ship captures the same optimum deterministically ($2.174\times$ geomean over the old serial default), so config-RL’s standing contribution is that RL reproduces a hand-derivable heuristic. **edit-RL** (the policy emits source edits graded through the full untrusted battery) is a negative result: over 320 attempts, 208 (65%) of edits did not apply, because the model could not reproduce exact source lines; 52 applied and passed every gate but ran no faster; 37 applied but broke correctness and were caught by the gates; 19 failed at execution; and 4 failed inside the sandbox for reasons not attributable to the edit. The maximum speedup was $0.183\times$ (roughly $5\times$ slower) and no edit beat the baseline. The verifier rejected every incorrect edit. Source generation is the weakest action space for pointing a policy at this problem, and the result is credible *because* of how strictly it was judged.

C.5 Speed baselines and killed levers

We record the Triton kernels’ speed against three baselines, kept strictly separate so a ratio is never ambiguous, and we state the direction of each. Against the #904-crippled official Mamba-3 backward we are about $4\times$ slower (0.22 to $0.36\times$), a structural cost of avoiding `t1.dot` to survive #904; the real win over that baseline is *availability* rather than latency: our kernels compile at every `num_warps` setting where the official one fails. Against healthy Mamba-1 SSD CUDA we range from slower to

near-parity on the forward (0.21 to 1.04 $\times$, beating it only at the largest shape) and are slower on the backward (0.13 to 0.75 $\times$). Against our *own* earlier default a shape-gated selector is 2.174 $\times$ faster (geomean); this is internal autotuning, not a claim of external speed. Separately, four optimizations were built or probed and measured to give no benefit. We record each with its mechanism so that it is not attempted again: a K#1 two-level scan (1.11 $\times$ kernel-only, below the 1.3 $\times$ gate, because halving the step count is cancelled by a 3 $\times$ -wider carry); a stage-B GEMM fusion (a measured negative, fusible ceiling 0.57 ms or 10% of the save-forward time at $d_v=128$); activation-checkpointing for the C6 backward (zero speedup, reverted); and an archived CUDA warp-scan experiment kept for the record.

References

- [1] Tri Dao and Albert Gu. Transformers are SSMs: Generalized models and efficient algorithms through structured state space duality. In *International Conference on Machine Learning (ICML)*, 2024. arXiv:2405.21060.
- [2] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.
- [3] Aakash Lahoti, Kevin Y. Li, Berlin Chen, Caitlin Wang, Aviv Bick, J. Zico Kolter, Tri Dao, and Albert Gu. Mamba-3: Improved sequence modeling using state space principles. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2603.15569; OpenReview id HwCvaJOiCj.
- [4] Robert Tjarko Lange, Qi Sun, Aaditya Prasad, Maxence Faldor, Yujin Tang, and David Ha. Towards robust agentic CUDA kernel benchmarking, verification, and optimization. *arXiv preprint arXiv:2509.14279*, 2025.
- [5] Jianling Li, Shangzhan Li, Zhenye Gao, Qi Shi, Yuxuan Li, Zefan Wang, Jiacheng Huang, Haojie Wang, Jianrong Wang, Xu Han, Zhiyuan Liu, and Maosong Sun. TritonBench: Benchmarking large language model capabilities for generating Triton operators. *arXiv preprint arXiv:2502.14752*, 2025.
- [6] Xiaoya Li, Albert Wang, Guoyin Wang, Jiwei Li, and Chris Shum. CUDA-L1: Improving CUDA optimization via contrastive reinforcement learning. *arXiv preprint arXiv:2507.14111*, 2025.
- [7] Wei Liu, Jiawei Xu, Yingru Li, Longtao Zheng, Tianjian Li, Qian Liu, and Junxian He. Dr. Kernel: Reinforcement learning done right for Triton kernel generations. *arXiv preprint arXiv:2602.05885*, 2026. Environment KernelGYM; dataset hkust-nlp/drkernel-coldstart-8k.
- [8] Anne Ouyang, Simon Guo, Simran Arora, Alex L. Zhang, William Hu, Christopher Ré, and Azalia Mirhoseini. KernelBench: Can LLMs write efficient GPU kernels? *arXiv preprint arXiv:2502.10517*, 2025.
- [9] Sakana AI. The AI CUDA engineer archive. <https://huggingface.co/datasets/SakanaAI/AI-CUDA-Engineer-Archive>, 2025. CC BY 4.0; released February 2025 with *The AI CUDA Engineer*; revision 4edbe8d.
- [10] Dipankar Sarkar. The correctness illusion in LLM-generated GPU kernels. *arXiv preprint arXiv:2606.20128*, 2026.
- [11] Dipankar Sarkar. Operator-aware mixed-precision tolerance calibration for tensor kernels. *arXiv preprint arXiv:2607.16228*, 2026.
- [12] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024. Introduces Group Relative Policy Optimization (GRPO).

- [13] toleybot. Issue #904: Mamba3 backward pass (mamba3_8iso_bwd_kernel_dqkv) 38.7x slower on GB200 (SM100) due to ptxas C7907 eliminating autotuner configs. <https://github.com/state-spaces/mamba/issues/904>, 2026. Open; companion Triton issue #9933, also open. The tensor-memory mitigation is Triton PR #9093, merged to main 2026-05-01 and not present in any 3.7.x release.
- [14] Cooper Veit. Kernel contracts: A specification language for ML kernel correctness across heterogeneous silicon. *arXiv preprint arXiv:2604.22032*, 2026.
- [15] Patrick Wagner, Nils Strodthoff, Ralf-Dieter Bousseljot, Dieter Kreiseler, Fatima I. Lunze, Wojciech Samek, and Tobias Schaeffter. PTB-XL, a large publicly available electrocardiography dataset. *Scientific Data*, 7(1):154, 2020.
- [16] Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated delta networks: Improving mamba2 with delta rule. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2412.06464.
- [17] Songlin Yang, Bailin Wang, Yikang Shen, Rameswar Panda, and Yoon Kim. Gated linear attention transformers with hardware-efficient training. In *International Conference on Machine Learning (ICML)*, 2024. arXiv:2312.06635.
- [18] Songlin Yang, Bailin Wang, Yu Zhang, Yikang Shen, and Yoon Kim. Parallelizing linear transformers with the delta rule over sequence length. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2406.06484.
- [19] Songlin Yang and Yu Zhang. FLA: A triton-based library for hardware-efficient implementations of linear attention mechanism. <https://github.com/fla-org/flash-linear-attention>, January 2024.
- [20] Ziqian Zhong, Ivgeni Segal, Ivan Bercovich, Shashwat Saxena, Kexun Zhang, and Aditi Raghunathan. Hardening agent benchmarks with adversarial hacker-fixer loops. *arXiv preprint arXiv:2606.08960*, 2026.